Field Hours

CSCE 463 Networks and Distributed Processing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Delay

A packet does not arrive instantly. The delay is processing at the nodes, plus transmission onto the link, plus propagation down the link, plus time spent waiting in queues. You can compute the first three from the size, the rate, and the distance. The queue is the one that depends on other people's traffic.

Transmission delay is the size divided by the bandwidth. Propagation is the distance divided by the speed in the medium. A large file on a fast long link is often transmission. A small packet on a slow long link can be propagation. Say which term you mean.

Four contributions.

Process -> Transmit: size / rate -> Propagate: distance / speed -> Queue

One packet, one link

1. A 1,000 byte packet. The link is 1 megabit per second. Transmission is 8,000 bits over 1,000,000 bits per second, which is 8 milliseconds.

2. The link is 300 kilometers of fiber, about 1 millisecond of propagation at a typical fiber speed. Use the number you were given in the problem. Do not mix it with the 8.

3. If the queue is empty and processing is negligible, the delay is about 9 milliseconds. A busy queue can dwarf both.

Problem 1. You doubled the bandwidth and the small control packet is still slow. Which term did you not buy down?

Answer. Propagation, and possibly processing. Bandwidth shrinks transmission delay. A 40 byte packet was already a tiny transmission. The distance did not change.

Problem 2. Why is this course outside the 36 credits if CSCE 612 is on the plan?

Answer. The catalog counts only one of CSCE 463 and CSCE 612 toward a degree. Here 463 is the preparation and 612 is the master's course. Do not list both inside the ten.

Lecture 2 · 50 minutes

A link

A link can corrupt bits. A checksum detects some corruption. It does not correct it, and a weak checksum misses some errors. An acknowledgement says the packet arrived intact. A timeout says you stopped waiting. Retransmission sends again. Together those three are a reliable link, in outline.

Stop-and-wait sends one packet and waits. The link sits idle during the wait. A window of several packets fills the pipe. The window is not free: the sender must remember what is unacknowledged. CSCE 612 will derive the window from the delay and the rate. Here, see why one-at-a-time is slow.

Stop-and-wait.

1. Send one packet

2. Wait for the acknowledgement

3. The pipe is idle during the wait

4. A timeout sends it again

A long fat link

1. Transmission of one packet is 1 millisecond. The round trip is 100 milliseconds.

2. Stop-and-wait spends about 1 millisecond sending and 100 waiting, per packet. The useful fraction is about 1 percent.

3. A window of about 100 packets would keep the sender busy. You have not built that protocol. You have seen the arithmetic that demands it.

Problem 1. The checksum passes and the packet is still wrong. Is that possible?

Answer. Yes. A checksum is a finite check. Some different bit patterns share a checksum. A stronger check lowers that chance and does not make it zero. Do not treat a pass as proof of perfection.

Problem 2. A timeout fires and the packet had arrived. What does the receiver see if you retransmit naively?

Answer. A duplicate. The receiver needs a sequence number, or an equivalent, to drop the second copy. Reliability without identity creates repeats. The graduate course will not let you forget that.

Lecture 3 · 50 minutes

Forwarding

Forwarding is what a router does with one packet: look up the destination in a table and send the packet out one link. The table is not the whole map of the internet. It is a set of prefixes. The longest matching prefix wins.

Routing is how the table gets built. Do not use the two words as synonyms. A bug in forwarding drops this packet. A bug in routing fills the tables wrong for everyone who trusts them.

Two jobs.

Forwarding

One packet, one table lookup

Happens at line rate

Routing

Messages between routers

Builds the table over time

Longest prefix

1. The table has 10.0.0.0/8 out of link A, and 10.1.0.0/16 out of link B.

2. A packet for 10.1.2.3 matches both. The longer prefix, /16, wins. The packet goes to B.

3. A packet for 10.2.2.3 matches only /8. It goes to A. If you had stopped at the first match, you might have sent 10.1.2.3 to A and missed the more specific route.

Problem 1. Two prefixes match and you pick the shorter one. What goes wrong?

Answer. Traffic that had a more specific route takes the general one. That can be a black hole, a detour, or a leak into the wrong network. Longest match is the rule. First match is an accident of table order.

Problem 2. The packet took the right link and the application still failed. Is that a forwarding bug?

Answer. Not by itself. Forwarding delivered the packet to the next hop. The failure may be later: the port, the loss, the server. Check the hop you meant before you blame the route.

Lecture 4 · 50 minutes

Routing in outline

Distance-vector: each router tells its neighbors the destinations it can reach and the cost. A neighbor adds the link cost and keeps the smallest. The simple version counts to infinity when a link dies, because a neighbor's old advertisement loops back. Split horizon and a poison reverse are patches. They are not a proof that loops are impossible.

Link-state: each router floods the state of its own links. Everyone computes the same shortest paths from that map. The flood must converge on one map. A partial map is a wrong map. CSCE 612 compares these carefully. You need the failure mode of the first one.

A dead link, distance-vector.

1. The route used to point at the neighbor

2. The neighbor still advertises a route that points back

3. The cost climbs instead of dying

4. That climb is count-to-infinity

Two routers

1. A reaches Z through B, cost 1 plus B's cost. B's link to Z dies.

2. Before B has a chance to say so honestly, B hears A's advertisement: A can reach Z. B now points at A. A still points at B.

3. Each round adds the link cost. The route should have been deleted. The advertisements formed a

loop. You will compute one round of this by hand in the problems.

Problem 1. Link-state floods every link to every router. What do you gain, and what do you spend?

Answer. Each router can run a shortest-path computation on a full map, so it does not trust a neighbor's summary. You spend flood traffic and you must deal with a map that is temporarily inconsistent. The protocol is not free.

Problem 2. Is count-to-infinity a forwarding bug?

Answer. No. Forwarding obeyed the table. The table was built by a routing exchange that looped. Fix the routing rule or the timers. Do not 'fix' it by dropping a random prefix.

Lecture 5 · 50 minutes

A reliable byte stream

An application often wants a stream of bytes, in order, without gaps or duplicates. The network underneath is packets that can be lost, reordered, or repeated. The transport above fills that gap with sequence numbers, acknowledgements, retransmission, and a window.

A connection is state at both ends about that stream. A handshake agrees to create the state. A finish agrees to release it. Skipping the handshake and hoping is how two sides disagree about whether the conversation exists. You are not implementing the full protocol. You are naming the state.

What the stream hides.

Bytes from the application -> Segments with sequence numbers -> Acks and retransmission -> Bytes, in order, at the other end

A lost segment

1. You sent bytes 1 through 100, then 101 through 200. The second segment is lost.

2. The receiver has 1 through 100. It does not hand the application a hole. It acknowledges 100 and waits.

3. The sender's timeout retransmits 101 through 200. The receiver then delivers both ranges, in order. The application never saw the loss.

Problem 1. The receiver got segment 200 before segment 100, and it delivered 200 to the application immediately. What broke?

Answer. The in-order promise. Hold 200 until 100 arrives, or the application must be written for datagrams, not a stream. Those are different interfaces. Do not mix them silently.

Problem 2. Both sides think the connection is open, and one has rebooted and forgotten. What was the handshake for?

Answer. To create state both sides remember. After a reboot the state is gone. Packets from the old conversation must not be accepted as part of a new one. Identifiers and the handshake are how you tell the two apart. The graduate course spends a week on this.

Lecture 6 · 50 minutes

Names and a request

People use names. Routers use addresses. A name system answers a name with an address. The answer can be cached, and a cache can be stale. A failed lookup is not the same as a refused connection.

A request on top of the byte stream names a resource and a method. The response is a code and a body. You can now read a packet capture at the level of delay, a prefix, a retransmission, and a request. CSCE 612 starts from that reading and asks you to design, not only to define.

One web fetch, bottom to top.

Addresses and prefixes

A byte stream

A name that became an address

A request and a response code

A page that fails

1. The name lookup fails. You never open a connection. The bug is not a timeout of the stream.

2. The lookup succeeds and the connection times out. The address was wrong, the packet was filtered, or the server is down. The name system did its job.

3. The connection succeeds and the response code is 404. The stream worked. The resource name did not. Three different failures. Write down which layer you blame before you change a setting.

Problem 1. A cached name points at yesterday's address. The host moved. What does the user see, and what expired?

Answer. A connection to the old address, which may refuse or be someone else. The cache lifetime expired in the world and not in the cache. Shorten the lifetime or flush it. Do not retransmit the old address harder.

Problem 2. What should you be able to compute before CSCE 612?

Answer. A transmission delay, a longest-prefix choice, and one round of a routing loop. The master's course will not rederive what a prefix is.